

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 5: **Eloquent ORM and Relationships**

Mapping Objects to Tables, Model Relationships, and Efficient Queries

What You'll Learn

1. Explain the **ORM** concept and how Eloquent maps classes to database tables
2. Define **relationships** between models (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`)
3. Understand **eager loading** and how to avoid the **N+1** problem
4. Apply **pagination** to large-scale data listings

This slide deck covers concepts. Applying relationships, eager loading, and pagination to Simple POS happens in the practicum jobsheet.

Part 1

Without a Relationship vs. With an Eloquent Relationship

Without an Eloquent relationship

- Every time related data is needed, hand-write a join query again
- Prone to typos in join column names, repeated in many places

With an Eloquent relationship

- Declared once, in the model
- Used anywhere through a property or method, no rewriting the join

Similar to a family tree: the line of descent gets drawn once, and anyone reading that tree instantly understands the relationship, no digging through separate records.

The ORM Concept: Mapping Classes to Tables

ORM (Object-Relational Mapping): a technique that maps a class in code to a table in the database, and an object to a row, so database operations are written through methods and object properties instead of raw SQL queries.



- Eloquent uses the **Active Record** pattern: one model class both represents a table AND provides the methods to query, save, and delete its own data

Relationships: Connecting Models

Relationship: a method on an Eloquent model describing how one table connects to another, used like an ordinary property even though it's really running a query behind the scenes.

```
class Author extends Model
{
    public function articles(): HasMany
    {
        return $this->hasMany(Article::class);
    }
}

class Article extends Model
{
    public function author(): BelongsTo
    {
        return $this->belongsTo(Author::class);
    }
}
```

authors (one)



articles (many)

hasOne: A One-to-One Relationship

```
class Author extends Model
{
    public function profile(): HasOne
    {
        return $this->hasOne(Profile::class);
    }
}
```

- Similar to `hasMany`, but guarantees only **one** related row, not many
- Example: one author has one profile, not several

belongsToMany: A Many-to-Many Relationship

```
class Article extends Model
{
    public function tags(): BelongsToMany
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

- One article can have many tags, one tag can be used by many articles
- Needs a pivot table in between, e.g. `article_tag`, following the convention: both singular model names, alphabetical order, joined by an underscore

Four Relationship Types: A Summary

Relationship	Meaning	Example
hasOne	One parent row has one related row	Author has one Profile
hasMany	One parent row has many related rows	Author has many Articles
belongsTo	This row points back to one parent row	Article points to one Author
belongsToMany	Many-to-many through a pivot table	Article and Tag connect both ways

A Relationship's Name Isn't Just Cosmetic

- That name becomes the key used in three different places:
 - **Method call** `$author->articles()` : returns a query builder, chainable with `->where()` , `->orderBy()` , etc.
 - **Property access** `$author->articles` : returns the loaded collection directly
 - **String inside** `with('articles')` : tells Eloquent which relationship to load up front

Part 2

The N+1 Problem

Lazy Loading: a relationship only loads once that property is actually accessed, row by row, the root cause of the N+1 problem.

N+1: a problematic query pattern, one query for the main listing, plus one separate query for every single row to load its relationship.

- Displaying 15 articles with each one's author name, without eager loading: 1 list query + 15 author queries = **16 queries**

Approach	Query Count
Lazy loading (no <code>with()</code>)	1 list query + N relationship queries (N+1)
Eager loading (<code>with('author')</code>)	2 queries total

Eager Loading

Eager Loading: loading a relationship up front through `with()` , in a fixed, small number of queries, instead of loading it one row at a time only when it's needed.

```
$articles = Article::with('author')  
    ->latest()  
    ->paginate(15);
```

The query count under eager loading doesn't depend on how many rows get displayed, only on how many relationship levels get loaded. Displaying 15 articles or 150 articles with `with('author')` both cost 2 queries total, not 2 multiplied by the row count.

Nested Eager Loading: Dot Notation

```
$authors = Author::with('articles.comments')->get();
```

- Loads every author, their `articles` relationship, **and** the `comments` relationship inside each article, all at once
- Still a fixed query count, not growing with the number of rows

Part 3

Pagination

Pagination: a technique for splitting a large query result into small pages, returning only a subset of rows at a time along with metadata about the total page count.

```
public function index()
{
    $articles = Article::with('author')
        ->orderBy('title')
        ->paginate(10);

    return view('articles.index', compact('articles'));
}
```

```
<!-- resources/views/articles/index.blade.php -->
<div class="mt-4">
    {{ $articles->links() }}
</div>
```


When to Use Pagination

Reach for `paginate()` whenever a row count could keep growing without bound over time, like an article listing or a comment history. A plain `get()` is still fine for a listing that's genuinely small and stays small, like a tag dropdown for a filter.

Proving Pagination Works

```
php artisan tinker
>>> Article::orderBy('title')->paginate(10)->pluck('id');
>>> Article::orderBy('title')->paginate(10, ['*'], 'page', 2)->pluck('id');
```

- Compare the results of both commands
- Overlapping IDs between page 1 and page 2 signal a problem with the `orderBy` or the data

Applying This to Simple POS

- You'll apply the relationship, eager loading, and pagination concepts directly to the Simple POS case study in the practicum jobsheet
- Mapping the `Category - Product` and `Transaction - TransactionDetail - Product` relationships, avoiding N+1 on a 2,500-transaction listing, and proving pagination across 300 products

Full code: github.com/se-polinema/simple-pos-ch05

Summary

- Relationships (`hasOne` , `hasMany` , `belongsTo` , `belongsToMany`) are declared once in a model, then used through a method call, property access, or a string inside `with()`
- Lazy loading triggers one extra query per row per relationship (the N+1 problem); eager loading, including nested dot notation, loads a relationship in a fixed query count, independent of row count
- An ORM like Eloquent (using the Active Record pattern) maps classes to tables and objects to rows, replacing raw SQL queries with object methods and properties
- `paginate()` splits a large listing into small pages; different pages can be proven to genuinely differ by comparing result IDs across pages

References & Discussion

Official Laravel documentation (Eloquent, Relationships, Pagination)

Full code: `github.com/se-polinema/simple-pos`

Next meeting: Input Validation & Security